

Plan: Neural SC Decoder for MAC Polar Codes – All Classes, All Channels

Goal

A neural decoder that: - Achieves BLER within $\sim 1.5\times$ of analytical SC - Works for all code classes (A, B, C, and arbitrary intermediate paths) - Works for all channels (GMAC, BEMAC, ABNMAC, ISI-MAC) - Has complexity $O(N \log N \times \text{NN_size})$ – same tree structure as SC, just with learned operations - Scales to $N=256$ and beyond

What we know works

Corner rates (**Class A** / Class C) – SOLVED

- Two chained single-user NPDs
- fast_ce training, $O(\log N)$ gradient depth
- NPD-guided frozen set design (MI-based)
- Result: **beats SC at $N=16-256$ on GMAC**
- No CalcParent needed (tree only goes down)

Non-corner rates (Class B / arbitrary paths) – NOT SOLVED

- Production CG decoder (nec_gmac.py) achieves $3.3\times$ SC at $N=256$
- Sequential training only ($O(N \log N)$ gradient depth) – 28 hrs at $N=256$
- Gap is from autoregressive error cascade, not per-position MI deficit
- NPD-guided frozen set design doesn't help (MI >0.98 everywhere under teacher forcing)
- The 4-class NPD with fast_ce gets $5.8\times$ SC at $N=32$ (exposure bias)

The fundamental problem for non-corner rates

Non-corner paths (e.g., Class B = $0^{\{N/2\}} 1^N 0^{\{N/2\}}$) require CalcParent – a bottom-up operation that propagates information UP the tree when the path switches between users. This makes: 1. fast_ce impossible (fast_ce is top-down only) 2. Training sequential and slow 3. The decoder autoregressive with error cascade risk

Strategy: bridge from what works (corners) to what doesn't (non-corners)

Phase 1: Understand what the NPD paper actually does for different channels

- ☐ Study how the NPD paper handles channels with memory (ISI, trapdoor)
 - They use the same fast_ce framework
 - They use trellis-based tree operations for memory channels
 - What is their CheckNodeTrellis / BitNodeTrellis?
- ☐ Study how the NPD paper does code design for unknown channels
 - estimation_step + improvement_step alternation
 - MI-based position selection
 - Does this generalize to MAC?
- ☐ Determine: does the NPD paper ever handle multi-user / MAC?
 - If not, we are extending their framework

Phase 2: Can we make fast_ce work for non-corner paths?

- ☐ The blocker is CalcParent (bottom-up). Analyze exactly when CalcParent is needed:
 - Class B path $0^{\{N/2\}} 1^N 0^{\{N/2\}}$: CalcParent at the boundary between phases
 - How many CalcParent operations per codeword? At which tree depths?
 - Is it a small fraction of total ops or dominant?

- Option 2A: “Almost fast_ce” – run fast_ce top-down for the majority of ops, handle CalcParent boundaries separately
 - Gradient depth would be $O(\log N)$ for the parallel parts + $O(K)$ for the CalcParent boundaries
 - If K is small (e.g., $K = \log N$), this is still $O(\log N)$ overall
- Option 2B: Reformulate CalcParent as a top-down operation
 - Can we pre-compute or approximate the bottom-up information?
 - Would a learned “shortcut” that predicts CalcParent output work?
- Option 2C: Avoid CalcParent entirely by restructuring the path
 - All paths in $0^i 1^N 0^{N-i}$ decompose into 3 single-user phases
 - The middle phase (V with partial U known) is the hard one
 - Can we train a single-user decoder for the “partially conditioned” channel?

Phase 3: Solve the middle phase of Class B

- The Class B middle phase: decode V when $N/2$ U -bits are known
 - After decoding first $N/2$ U -bits, reconstruct partial X
 - But polar encoding mixes ALL U -bits – knowing half of U doesn’t give half of X cleanly
 - This is why the 3-stage single-user approach failed in our POC
- Investigate: what does the partial X reconstruction actually look like?
 - For the known U positions, their contribution to X is deterministic
 - For the unknown U positions, their contribution is random
 - Can we compute per-position “how much X is known” and use that as a channel feature?
- Option 3A: Train the V decoder on the actual partially-conditioned channel
 - Generate training data where first $N/2$ U -bits are decoded (with some errors)
 - The V decoder sees z and the partial X as side information
 - $z_encoder$ takes $(z, \text{partial_x_hat})$ as 2D input per position
- Option 3B: Use the analytical partial-subtraction approach
 - Compute the LLR for V given partial X knowledge analytically
 - Feed this LLR to the NPD instead of raw z
 - Does the analytical LLR help even when X has errors?

Phase 4: The 4-class joint approach revisited

- The 4-class NPD with fast_ce got BLER=0.27 at $N=32$ with corrected decoder
 - This was with genie design – never tried with NPD-guided design
 - NPD-guided design might help the 4-class case just like it helped the binary case
- POC: train 4-class NPD at rate 1, measure 4-class MI, pick positions, evaluate
 - If this brings 4-class NPD from 5.8x SC to ~1.5x SC, it’s a viable path for Class B
 - Quick test at $N=16-32$ (~20 min)
- The 4-class exposure bias problem
 - In binary NPD: sign-flip symmetry makes errors “look like valid inputs”
 - In 4-class NPD: a wrong (u,v) creates a partially-wrong embedding
 - NPD-guided design might avoid positions where the 4-class exposure bias is worst
 - Test: measure per-position 4-class MI under sequential decode vs teacher forcing

Phase 5: Production CG decoder with NPD-style methodology

- The production CG decoder already works at $N=256$ (BLER=0.017)
 - Trained with sequential TF on genie design
 - Never tried: rate-1 training + MI-guided design
 - Never tried: MI tracking during training
- The implications agent showed $MI > 0.98$ everywhere under teacher forcing
 - But this was with genie-rate training ($k_u=k_v=123$), not rate-1 training
 - Rate-1 training might reveal different per-position patterns

- Even if per-position MI is still high, the MI TRAJECTORY during training could reveal when/where the model stops improving
- Key experiment: train production CG on Class B at N=128 with:
 - Rate 1 (all info)
 - MI tracking every eval step
 - Compare to NPD’s MI trajectory on Class C
 - Takes ~14 hrs (sequential training)
- If MI-guided design helps CG at N=128, scale to N=256

Phase 6: Complexity and efficiency

- Current training costs:
 - NPD fast_ce at N=256: ~1 hr for 100K iters
 - CG sequential at N=256: ~28 hrs for 100K iters
 - Can we reduce CG training time?
- Options for faster CG training:
 - Curriculum: train at small N, transfer to large N (already proven for NPD)
 - Gradient checkpointing: reduce memory, same gradient depth
 - Mixed precision: faster matmuls
 - C++ tree walk extension: 1.34x speedup (already exists in project)
- Inference complexity comparison:
 - Analytical SC: $O(N \log N)$ scalar ops
 - NPD: $O(N \log N)$ MLP calls, each $O(d \times \text{hidden})$
 - CG: $O(N \log N)$ MLP calls, each $O(d \times \text{hidden})$, slightly larger MLPs
 - All are $O(N \log N \times \text{NN_size})$ – acceptable

Priority ranking

1. **Phase 4 POC** (4-class NPD + NPD-guided design at N=32) – 20 min, quick test of whether NPD-guided design fixes the 4-class problem
2. **Phase 2 analysis** (how many CalcParent ops does Class B actually need?) – 1 hr, determines if “almost fast_ce” is viable
3. **Phase 3A POC** (V decoder with partial X side info at N=16) – 30 min, tests the 3-stage decomposition properly
4. **Phase 5 experiment** (CG rate-1 training at N=128) – 14 hrs, the definitive CG test
5. **Phase 2A implementation** (“almost fast_ce” for Class B) – depends on Phase 2 analysis

Success criteria

- Class C: BLER $\leq 1.5 \times$ SC at N=16-256 [DONE] (ALREADY ACHIEVED)
- Class B: BLER $\leq 1.5 \times$ SC at N=32 (first milestone)
- Class B: BLER $\leq 1.5 \times$ SC at N=256 (final goal)
- Training time: ≤ 4 hrs at N=256 (practical)
- Works on GMAC, BEMAC, and at least one memory channel